

AWS 서버리스 실습: 투자 제안 심사 시스템

실습 개요

이 실습에서는 AWS 서버리스 아키텍처를 활용하여 간단한 투자 제안 심사 시스템을 구현합니다. 서버리스는 서버를 직접 관리하지 않아도 되는 컴퓨팅 방식으로, AWS Lambda가 필요할 때만 코드를 실행하고 자동으로 확장해 줍니다 ¹ ². 이 시스템에서는 API Gateway를 통해 Lambda 함수를 호출하고, 투자 제안의 점수를 평가하여 결과를 Amazon DynamoDB에 저장한 뒤 Amazon SNS로 관리자에게 알림을 보냅니다. 즉, **트리거(API 호출) → 처리(심사) → 저장(데이터베이스) → 알림(SNS)**의 흐름을 경험하게 됩니다. 참고로 AWS 튜토리얼에서도 API Gateway → Lambda → DynamoDB → SNS 구조의 예시가 소개되어 있습니다 ³. 실습 시간은 약 30분이며, 복잡한 코드보다는 흐름을 이해하는 데 중점을 둡니다.

아키텍처 개요

- **API Gateway**: 클라이언트(예: 웹 폼 또는 HTTP 클라이언트)로부터 HTTP 요청을 받아 **AWS Lambda** 함수로 전달합니다. API Gateway는 입력 데이터를 전달하고 Lambda의 응답을 클라이언트에 반환합니다 ⁴.
- **AWS Lambda**: 요청을 받아 제안 내용을 평가하는 코드를 실행합니다. Lambda는 필요한 시점에만 코드를 실행하고 자동으로 확장되므로 서버를 직접 관리할 필요가 없습니다 ¹ ².
- **Amazon DynamoDB**: NoSQL 데이터베이스 서비스로, 투자 제안의 평가 결과(예: '적격/부적격' 등)를 저장합니다. DynamoDB는 완전관리형(NoSQL) 데이터베이스로 빠른 성능과 무제한 확장성을 제공합니다 ⁵.
- **Amazon SNS**: Simple Notification Service로, Lambda에서 저장된 결과를 토대로 관리자에게 이메일이나 SMS 등의 알림을 보냅니다. SNS는 Pub/Sub 방식의 메시징 서비스로, 주제를 통해 다수의 구독자에게 메시지를 전달할 수 있습니다 ⁶.

이러한 서비스 연계를 통해, 예를 들어 “HTTP API 요청 → Lambda 코드 실행 → DynamoDB에 저장 → SNS로 알림”의 서버리스 흐름을 구성할 수 있습니다. AWS 공식 예제에서도 이와 유사한 방식으로 API Gateway와 Lambda를 이용해 DynamoDB를 조회하고 SNS로 알림을 전송하는 구조를 보여줍니다 ³.

사용 서비스 요약

- **AWS Lambda** - 코드 실행 서비스: 서버리스 컴퓨팅을 지원하여 사용자는 코드에만 집중하고 인프라 관리는 AWS가 맡습니다 ² ¹.
- **Amazon API Gateway** - HTTP API 엔드포인트: HTTP 요청을 받아 Lambda 함수를 호출해 주며, 외부 요청을 관리하고 라우팅합니다 ⁴.
- **Amazon DynamoDB** - NoSQL 데이터 저장소: 완전관리형 NoSQL DB로, 심사 결과(제안ID, 투자자명, 점수, 판단 결과 등)를 빠르게 저장/조회합니다 ⁵.
- **Amazon SNS** - 알림 서비스: Lambda에서 처리된 결과를 토픽으로 발행하여 구독된 관리자 이메일이나 SMS로 알림을 전송합니다 ⁶.
- **(선택) Amazon S3** - 파일 저장소: 필요에 따라 투자 제안 파일(문서, 이미지 등)을 S3에 업로드하고, 업로드 이벤트를 Lambda 트리거로 사용할 수 있습니다.

각 서비스는 AWS 관리 콘솔에서 클릭 몇 번으로 생성할 수 있으며, 실습 안내에 따라 순서대로 구성하면 됩니다.

단계별 실습 가이드

1단계: DynamoDB 테이블 생성 (약 5분)

- **목표:** 투자 제안 심사 결과를 저장할 DynamoDB 테이블을 생성합니다.
- **설명:** AWS 콘솔에서 **DynamoDB** 서비스로 이동하여 “테이블 생성”을 클릭합니다. 테이블 이름은 예를 들어 `Proposals` 로 지정하고, **기본 파티션 키**를 `proposal_id` (문자열 타입)로 설정합니다. 나머지 설정은 기본값을 사용합니다. 테이블이 생성되면, Lambda 함수에서 `PutItem` API로 제안 데이터를 저장할 수 있습니다. (예: `proposal_id`, `name`, `score`, `status` 컬럼을 저장)

2단계: SNS 주제 및 구독 생성 (약 5분)

- **목표:** 결과 알림용 SNS 주제를 만들고 관리자(본인) 이메일로 구독합니다.
- **설명:** AWS 콘솔에서 **SNS** 서비스로 이동하여 “주제(Topic)”를 생성합니다. 예를 들어 `ProposalAlerts` 라는 이름으로 생성합니다. 주제가 만들어지면 해당 주제의 ARN(리소스 이름)을 복사해 둡니다. 이어서 구독을 추가하여 이메일 주소를 등록합니다. 이메일 구독 시 AWS에서 확인 메일이 발송되므로, 메일의 링크를 클릭해 구독을 확인(승인)합니다. (이 과정을 통해 “홍길동 님의 제안이 적격 판정되었습니다.”와 같은 메시지를 받을 준비가 됩니다.) SNS 주제 생성 후에는 Lambda 코드에서 이 주제의 ARN을 사용하여 알림을 전송하게 됩니다.

3단계: Lambda 함수 생성 및 코드 작성 (약 10분)

- **목표:** 투자 제안을 평가하고 저장·알림을 수행하는 Lambda 함수를 작성합니다.
- **설명:** AWS Lambda 콘솔로 이동하여 “함수 생성”을 클릭합니다. 함수 이름을 예를 들어 `ProposalReviewFunction` 으로 지정하고, 런타임은 **Python 3.x** (또는 Node.js 등) 을 선택합니다. 실행 역할(permissions role)은 기본권한에 `DynamoDB` 와 `SNS` 접근 권한이 포함되어야 합니다. (필요시 콘솔 안내에 따라 “기존 역할 사용”을 선택하고, 이전 단계에서 만든 DynamoDB, SNS에 접근 가능한 역할을 부여합니다.)

Lambda 코드 예시 (Python):

```
import json
import boto3

def lambda_handler(event, context):
    # 1. 이벤트 데이터 파싱 (API Gateway를 통해 전달된 JSON)
    data = json.loads(event['body'])
    proposal_id = data['proposal_id']
    name = data['name']
    score = int(data['score'])

    # 2. 심사 로직: 점수가 75 이상이면 '적격', 그렇지 않으면 '부적격'
    status = "적격" if score >= 75 else "부적격"

    # 3. DynamoDB에 결과 저장
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table('Proposals')
    table.put_item(Item={
        'proposal_id': proposal_id,
        'name': name,
        'score': score,
```

```

        'status': status
    })

# 4. SNS로 관리자에게 알림 (생성한 SNS 주제 ARN 사용)
sns = boto3.client('sns')
message = f"{name}님의 투자 제안이 {status} 판정되었습니다."
sns.publish(
    TopicArn='arn:aws:sns:ap-northeast-2:123456789012:ProposalAlerts', # 자신의 SNS 주제
    # ARN으로 변경
    Message=message
)

# 5. API 응답 반환
return {
    'statusCode': 200,
    'body': json.dumps({'result': status})
}

```

위 코드에서 DynamoDB 테이블 이름('Proposals')과 SNS Topic ARN('arn:aws:sns:...:ProposalAlerts')은 자신이 생성한 리소스 이름/ARN으로 변경해야 합니다. 코드를 작성한 후 저장합니다. 이 Lambda 함수는 API 요청으로부터 제안 정보를 받아 score 값을 기준으로 심사를 하고, 결과를 DB에 저장하며 SNS로 알림을 발송합니다.

4단계: API Gateway 설정 (약 5분)

- **목표:** API Gateway를 생성하여 Lambda 함수를 호출하는 HTTP 엔드포인트를 만듭니다.
- **설명:** AWS 콘솔에서 **API Gateway** 서비스로 이동합니다. **HTTP API**(또는 REST API)를 생성하고, 새 경로(/proposals 등)와 메서드(POST)를 설정합니다. 통합 대상(Integration)으로 **Lambda**를 선택하고, 앞서 만든 ProposalReviewFunction 을 연결합니다. 생성된 API의 Invoke URL(호출용 URL)을 복사해 둡니다. (예시: https://{api-id}.execute-api.ap-northeast-2.amazonaws.com/proposals) 이렇게 하면 클라이언트의 HTTP 요청이 API Gateway를 통해 Lambda 함수에 전달됩니다

4

5단계: 테스트 및 결과 확인 (약 5분)

- **목표:** 전체 흐름이 정상 동작하는지 확인합니다.
- **설명:** 복사한 API Gateway 호출 URL을 사용하여 HTTP POST 요청을 보냅니다. 예시로, 터미널(curl)이나 Postman 등을 이용해 JSON 데이터를 전송합니다.

```

curl -X POST -H "Content-Type: application/json" \
  -d '{"proposal_id":"P100","name":"홍길동","score":85}' \
  https://{API_ID}.execute-api.ap-northeast-2.amazonaws.com/proposals

```

이 요청으로 Lambda 함수가 실행되고, 점수 85를 기준으로 '적격' 판정되어 DynamoDB에 저장됩니다. 성공적으로 실행되면 응답으로 {"result": "적격"} 같은 결과가 반환됩니다. DynamoDB 콘솔에서 Proposals 테이블을 열어 해당 항목이 저장되었는지 확인합니다. 또한, SNS 구독 이메일을 확인하여 “홍길동님의 투자 제안이 적격 판정되었습니다.”라는 알림 메시지가 도착했는지도 체크합니다. (이메일 수신은 SNS 구독 확인 후 약간의 시간이 걸릴 수 있습니다.)

결과 확인 및 마무리

모든 단계가 완료되면, API Gateway를 통한 호출→Lambda 실행→DynamoDB 저장→SNS 알림 전송의 서버리스 흐름이 정상적으로 동작함을 확인할 수 있습니다. 실습을 통해 AWS Lambda 기반 서버리스 처리 과정(트리거 → 로직 실행 → 결과 저장 → 알림 발송)을 직접 경험했습니다. 필요 시 다른 입력값(예: `score`)을 변경)으로 여러 번 실습해 보며 흐름을 확인하고, AWS 관리 콘솔에서 로그와 테이블, SNS를 살펴보면 이해에 도움이 됩니다. 이로써 약 30분 내에 AWS 서버리스 기초 요소들을 활용해 간단한 투자 제안 심사 시스템을 구성해보는 실습이 마무리되었습니다.

참고: AWS Lambda는 코드 실행에만 집중할 수 있게 해주며 자동 확장/과금으로 운영 부담을 줄여줍니다 ¹ ². API Gateway는 HTTP 요청을 Lambda로 라우팅하고 응답을 반환합니다 ⁴. DynamoDB는 완전관리형 NoSQL DB로 빠른 읽기/쓰기를 지원하며 ⁵, SNS는 다수 구독자에게 메시지를 전달하는 Pub/Sub 알림 서비스입니다 ⁶. 이러한 서비스들을 조합한 구조는 AWS 서버리스 예제에서도 자주 등장합니다 ³. 각 서비스 콘솔 화면에서는 직관적인 설정 단계를 제공하므로, 안내에 따라 차례대로 구성하면 초보자도 손쉽게 따라할 수 있습니다.

¹ ⁴ Get started with API Gateway - Amazon API Gateway

<https://docs.aws.amazon.com/apigateway/latest/developerguide/getting-started.html>

² Serverless Computing - AWS Lambda - Amazon Web Services

<https://aws.amazon.com/lambda/>

³ Use API Gateway to invoke a Lambda function - Amazon DynamoDB

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/example_cross_LambdaAPIGateway_section.html

⁵ Amazon DynamoDB - Choosing an AWS NoSQL Database

<https://docs.aws.amazon.com/whitepapers/latest/choosing-an-aws-nosql-database/amazon-dynamodb.html>

⁶ Create a notification alert system with AWS SNS, DynamoDB and AWS Lambda | by Harsha Siriwardana | Medium

<https://harshasiri.medium.com/create-a-notification-alert-system-with-aws-sns-dynamodb-and-aws-lambda-676afe2d8094>